Lab 1: Python and JupyterLab

Objective: We learn how to get started in using Python, which is widely used in scientific computation, machine learning, and quantum computer programming. In this lab, we learn to use an interactive environment in which you can write and execute simple Python codes.

What will be covered?

- Installation of Anaconda
- JupyterLab -- the interactive environment
- Example on data compression
- Installing QuTiP

1. Installing and Running Anaconda

Anaconda offers the easiest way to perform Python/R data science and machine learning on a single machine. The easiest way to install is to follow instructions at https://docs.anaconda.com/anaconda/install/.

After you install Anaconda, you can choose to run python in one of the following ways. The recommended way is JupyterLab, an interactive, browser-based environment.

IPython Interpreter:

- Open a terminal:
  - On macOS, open a **Terminal** from the **Launchpad**.
  - On Windows, open the **Anaconda Command Prompt** from the start menu.
- Now type *ipython*, then press *Enter* (or *Return*). You can test-drive the IPython Interpreter in interactive mode:

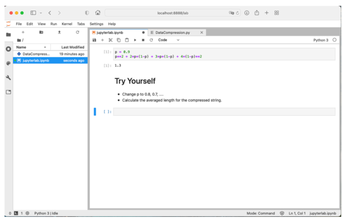
```
Python 3.7.3 (default, Mar 27 2019, 16:54:48)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.6.1 -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

- You can exit the interactive mode by typing *exit*, followed by pressing *Enter*.

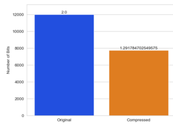
JupyterLab:

- The Anaconda Python also comes with the **Jupyter Notebook**--an interactive, browser-based environment in which you can write and execute code. For that, you use the **JupyterLab** interface (Read more at https://jupyterlab.readthedocs.io/en/latest/user/index.html).



Executing a Python script:

- You can also execute a python script (normally with the *.py* extension), e.g., Lab01_DataCompression.py, at the command line in a terminal by typing *ipython Lab01_DataCompression.py 6000 0.9*, then pressing *Enter*.



2. Playing with the Interactive Mode

Most of the functionality in Python is provided by **modules**. The Python Standard Library is a large collection of modules that provides cross-platform implementations of common facilities such as access to the operating system, file I/O, string management, network communication, and much more.

To use a module in a Python program it first has to be imported. A module can be imported using the import statement. For example, to import the module math, we can do:

In [ ]:
```
import math
```

In [ ]:
```
x = math.pow(2,50)
print(1e-6*x/60/60/24/365)
```

Note that the function `pow()` exists only in the namespace math, hence we need to include the prefix "math." as in `math.pow()` . Alternatively, we can chose to import all symbols (functions and variables) in a module to the current namespace:

In [ ]:
```
from math import *
x = pow(2,50)
print(1e-6*x/60/60/24/365)
```

Once a module is imported, we can list the symbols it provides using the *dir* function:

In [ ]:
```
import math
print(dir(math))
```

And using the function `help` we can get a description of each function (almost .. not all functions have docstrings, as they are technically called, but the vast majority of functions are documented this way).

In [ ]:
```
help(math.pow)
```

3. Data Compression, An Example

As we discussed in class, we consider classical information stored in the form of binary digits 0 (with probability p) and 1 (with probability $1-p$). We can develop a two-bit encoding scheme:

Original	Compressed	Length	Probability
00	0	1	p^2
01	10	2	$p(1-p)$
10	110	3	$p(1-p)$
11	1110	4	$(1-p)^2$

The average length of the compressed string can be easily calculated.

In [ ]:
```
def compressed_length(p=0.9):
    ''' Calculate the average length of the compressed string. '''
    l = p**2 + 2*p*(1-p) + 3*p*(1-p) + 4*(1-p)**2

    return l

print(compressed_length())
print(compressed_length(0.8))
print(compressed_length(0.7))
```

We can simulate the data compression and count the average length as follows.

In [ ]:
```
import random

def length_of_two_random_bits(prob):
    length = 1
    # Generate two bits according to prob
    if (random.random() > prob): length += 2 # bit 1
    if (random.random() > prob): length += 1 # bit 2

    return length

prob = 0.9
nbit_pairs = 6000
length_orig = 0
length_compressed = 0
for i in range(nbit_pairs):
    length_orig += 2
    length_compressed += length_of_two_random_bits(prob)
print(length_orig, length_compressed, length_compressed/nbit_pairs, compressed_length(prob))
```

4. Installing QuTiP

Examples in subsequent labs will rely heavily on QuTiP, which is open-source software for simulating the dynamics of open quantum systems. The QuTiP library depends on the excellent Numpy, Scipy, and Cython numerical packages. In addition, graphical output is provided by Matplotlib. QuTiP aims to provide user-friendly and efficient numerical simulations of a wide variety of Hamiltonians, including those with arbitrary time-dependence, commonly found in a wide range of physics applications such as quantum optics, trapped ions, superconducting circuits, and quantum nanomechanical resonators.

If you do not have qutip installed, check out https://qutip.org/docs/latest/installation.html#installing-with-conda.

We will learn to use it gradually in subsequent labs. If you are curious at this stage, please check out https://qutip.org/tutorials.html for tutorials.

References

1. Intro to Python for Computer Science and Data Science

P. Deitel and H. Deitel (Pearson, 2020). *Wonderful textbook for first-time Python learners from all educational background and majors.*

2. Learning Scientific Programming with Python

Christian Hill (Cambridge University Press, 2020)

https://scipython.com/book2/

This book is well illustrated and is supported by an extensive collection of resources online in the book's website, scipython.com. This site has code listings and solutions to exercises.

Lab 1 Exercises

Student Name: _____

Student ID: _____

Date: _____

E1. Sorting

Complete the code below to sort a list of random data in ascending order. Implement a compare-and-swap subroutine.

In [ ]:
```
import random

ndata = 20
list_data = []
for i in range(ndata):
    list_data.append(random.random())
```

In [ ]:
```
def compare_and_swap(j, k):
    ''' *** Implement your own code here *** '''

    return
```

In [ ]:
```
print(list_data)
```

In [ ]:
```
for j in range(ndata-1):
    for k in range(j+1, ndata):
        compare_and_swap(j, k)
```

You can exam the list of data below to confirm that your code works.

In [ ]:
```
print(list_data)
```

E2. Factoring Decision Problem

Given integer m and $l < m$, does m have a non-trivial factor less than l? Implement an algorithm to solve the problem.

In [ ]:
```
def factoring_decision(m, l):
    assert l < m

    ''' *** Implement your own code here *** '''

    return yes_or_no
```

To complete the problem, you are supposed to write down a few extra lines to test your code.

In [ ]: